

OPTION B INSTITUTE FAMILY

M13 - DELTA LAKE + LAKEHOUSE

Standalone Zero-to-Job-Ready Data Engineer 2026

BEGINNER-FIRST MASTER REPAIR RC4

11 canonical lessons | full 16-stage teacher contract | protected attempt-first review

This learner book is rebuilt from the frozen canonical technical material. It is not a front-matter patch: every lesson is re-laid out into the shared institute teaching grammar while preserving the course competency, source route, guided work, independent proof and validation boundary.

This book is your teacher

Use the book in order. External videos and official docs are visual companions, not hidden prerequisites. Watch or read only the assigned scope, then close it before the independent task.

The 16-stage lesson loop

Why it matters -> What you learn -> Prerequisite -> Picture it -> New words -> Useful source -> Follow once -> Expected result -> Why it worked -> Beginner mistakes -> Recovery -> Independent task -> Validate -> Teach it back -> Protected review + changed retry -> Evidence/mastery + exact next route.

Attempt-first protection

Never open a protected lesson review before saving a genuine independent attempt. If repair is needed, compare only the weak idea, close the review, solve a changed retry, then validate again. A solution seen during review is repair evidence, not mastery.

What counts as mastery

A reproducible artifact or explanation, an independent validation control, preserved failure/recovery evidence, a changed transfer task completed without the worked answer open, and an own-words explanation. Percent-complete is not a substitute.

How to use your devices

Phone: assigned video/reference, reading, revision. Laptop/PC: every real command, query, file, workbook, code task, validation and evidence artifact. Install only what the current lesson asks for.

Contents / Index

This book is your teacher	2
Contents / Index	3
Prerequisite / tool map	4
Practice / review / Gate route	4
DE13-01 - Data Lake vs Warehouse vs Lakehouse	5
DE13-02 - Why plain files are not tables	7
DE13-03 - Transaction log, ACID and table versions	9
DE13-04 - Create, read, update and delete	11
DE13-05 - Schema enforcement and schema evolution	13
DE13-06 - Time travel and version history	15
DE13-07 - MERGE, upserts and idempotent incremental loads	17
DE13-08 - Slowly Changing Dimension Type 2 with Delta	19
DE13-09 - Bronze, Silver and Gold layer contracts	21
DE13-10 - File layout, compaction and table maintenance	23
DE13-11 - Lakehouse integration and debugging	25
Module mastery checklist + revision quick reference	27

Prerequisite / tool map

Operate Delta/lakehouse tables with ACID, MERGE, schema evolution, history and layer contracts.

Item	Rule
Entry prerequisite	Previous module/Gate evidence must be stable.
Primary device	PC/laptop for hands-on work; phone is support only.
External sources	Optional/supporting unless the lesson explicitly marks an official current reference.
Evidence	Module practical + fresh retest + saved evidence; no critical unresolved failure.

Practice / review / Gate route

Lesson teaching -> follow once -> independent attempt -> validate -> save evidence. Only after the genuine attempt may protected review open. If weak: targeted repair -> close review -> changed retry -> revalidate. Finish the module practical/retest route before any downstream Gate that depends on this module.

DE13-01 - Data Lake vs Warehouse vs Lakehouse

1. Why it matters

A pipeline that produces correct rows once can still be unsafe if retries duplicate records, writers race, schema drifts silently, history disappears unexpectedly, or maintenance changes table meaning. The engineering skill is to preserve both business correctness and operational evidence.

2. What you are learning

By the end of this lesson you can data lake vs warehouse vs lakehouse and prove the result with saved evidence rather than a plausible screenshot.

3. Prerequisite / starting state

Starting state: no prior technical knowledge is assumed beyond the module Start Here instructions.

4. Picture it in your head

A data lake stores flexible files cheaply; a warehouse provides managed tables and query-oriented governance; a lakehouse tries to combine open/low-cost storage with table semantics and analytical management. The useful question is not which label sounds modern.

5. New words before we use them

A data lake stores flexible files cheaply; a warehouse provides managed tables and query-oriented governance; a lakehouse tries to combine open/low-cost storage with table semantics and analytical management. The useful question is not which label sounds modern. Ask what guarantees the consumer needs: transactions, schema, history, reproducibility, access control, performance and tooling.

6. Useful video / source

Source: Databricks — Diving into Delta Lake: Transaction Log

Assigned scope: Use matching transaction-log / ACID / time-travel section

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

Draw three boxes: lake, warehouse, lakehouse. For each, write storage form, table guarantees, typical consumers, and one trade-off. Then place Parquet and Delta correctly: Parquet is a file format; Delta adds table metadata/transaction semantics around data files. Classify five scenarios as lake, warehouse, or lakehouse and justify by guarantees, not brand names.

8. Expected result / what you should see

Save the input grain, the operation you performed, at least one independent row/value/schema/history control, and the evidence file or command that proves it. If a control differs, diagnose before continuing.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

- Calling Parquet itself a Delta table.
- Assuming ACID proves business logic is correct.
- Using automatic schema evolution as permission for any upstream change.
- Treating time travel as permanent backup.

DE13-01 - Practice and proof

11. If you get stuck

- Restate intended table grain/state. 2) Identify the first version/layer where the control diverges. 3) Check
- keys/schema before rewriting code. 4) Reveal only the next structural hint. 5) Protected Review is permitted only after
- a saved attempt.

12. Small independent task

For an IoT telemetry workload, choose lake/warehouse/lakehouse and defend the choice using transactions, history and analytical access.

13. Validate

Save the input grain, the operation you performed, at least one independent row/value/schema/history control, and the evidence file or command that proves it. If a control differs, diagnose before continuing.

14. Teach it back

In 60-90 seconds explain data lake vs warehouse vs lakehouse. Include one failure mode and one validation you would show a teammate.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery. Changed retry from the canonical lesson: Changed domain: classify a medical-imaging repository architecture and state required table guarantees.

16. Evidence / mastery + exact next route

You can repeat the changed task without the lesson/review open, validation passes, and you can defend why the result is trustworthy. A percentage score does not infer mastery. Exact next route: DE13-02

DE13-02 - Why plain files are not tables

1. Why it matters

A pipeline that produces correct rows once can still be unsafe if retries duplicate records, writers race, schema drifts silently, history disappears unexpectedly, or maintenance changes table meaning. The engineering skill is to preserve both business correctness and operational evidence.

2. What you are learning

By the end of this lesson you can why plain files are not tables and prove the result with saved evidence rather than a plausible screenshot.

3. Prerequisite / starting state

Prerequisite: DE13-01 mastered with saved evidence.

4. Picture it in your head

A directory full of Parquet files can hold typed columnar data, but the files alone do not tell concurrent writers which set of files forms the committed table snapshot. A table layer needs metadata that says which files are active, what schema is valid, and which operation produced the current state.

5. New words before we use them

A directory full of Parquet files can hold typed columnar data, but the files alone do not tell concurrent writers which set of files forms the committed table snapshot. A table layer needs metadata that says which files are active, what schema is valid, and which operation produced the current state. That is why “Parquet folder” and “Delta table” are not interchangeable.

6. Useful video / source

Source: Databricks — Diving into Delta Lake: Transaction Log

Assigned scope: Use matching transaction-log / ACID / time-travel section

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

Use the normal simulator output. Compare version_0.json with the input CSV files. Identify the extra table-state information: version, operation, schema and active rows. Explain why a consumer needs a committed snapshot rather than a random directory listing. Inspect base files and version_0 table snapshot; list three guarantees the snapshot metadata adds.

8. Expected result / what you should see

Save the input grain, the operation you performed, at least one independent row/value/schema/history control, and the evidence file or command that proves it. If a control differs, diagnose before continuing.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

- Calling Parquet itself a Delta table.
- Assuming ACID proves business logic is correct.
- Using automatic schema evolution as permission for any upstream change.
- Treating time travel as permanent backup.

DE13-02 - Practice and proof

11. If you get stuck

- Restate intended table grain/state. 2) Identify the first version/layer where the control diverges. 3) Check
- keys/schema before rewriting code. 4) Reveal only the next structural hint. 5) Protected Review is permitted only after
- a saved attempt.

12. Small independent task

Explain why copying a new Parquet file into a table directory is not equivalent to a committed table write.

13. Validate

Save the input grain, the operation you performed, at least one independent row/value/schema/history control, and the evidence file or command that proves it. If a control differs, diagnose before continuing.

14. Teach it back

In 60-90 seconds explain why plain files are not tables. Include one failure mode and one validation you would show a teammate.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery. Changed retry from the canonical lesson: Changed files: explain why a folder of versioned CSV exports still lacks transactional table semantics.

16. Evidence / mastery + exact next route

You can repeat the changed task without the lesson/review open, validation passes, and you can defend why the result is trustworthy. A percentage score does not infer mastery. Exact next route: DE13-03

DE13-03 - Transaction log, ACID and table versions

1. Why it matters

A pipeline that produces correct rows once can still be unsafe if retries duplicate records, writers race, schema drifts silently, history disappears unexpectedly, or maintenance changes table meaning. The engineering skill is to preserve both business correctness and operational evidence.

2. What you are learning

By the end of this lesson you can transaction log, acid and table versions and prove the result with saved evidence rather than a plausible screenshot.

3. Prerequisite / starting state

Prerequisite: DE13-02 mastered with saved evidence.

4. Picture it in your head

Delta records table changes in an ordered transaction log. A successful commit creates a new version; readers reconstruct a consistent snapshot from committed actions rather than seeing a half-finished multi-file write.

5. New words before we use them

Delta records table changes in an ordered transaction log. A successful commit creates a new version; readers reconstruct a consistent snapshot from committed actions rather than seeing a half-finished multi-file write. ACID is about transaction behavior, not about making bad business logic correct.

6. Useful video / source

Source: Databricks — Diving into Delta Lake: Transaction Log

Assigned scope: Use matching transaction-log / ACID / time-travel section

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

Open normal/log. Read versions 0 through 3. For each, state operation and row/value control. Show that version numbers are immutable history in the simulator. Then explain what real Delta would add beyond this classroom JSON model. Read commit logs 0-3 and explain atomic committed-version visibility.

8. Expected result / what you should see

Save the input grain, the operation you performed, at least one independent row/value/schema/history control, and the evidence file or command that proves it. If a control differs, diagnose before continuing.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

- Calling Parquet itself a Delta table.
- Assuming ACID proves business logic is correct.
- Using automatic schema evolution as permission for any upstream change.
- Treating time travel as permanent backup.

DE13-03 - Practice and proof

11. If you get stuck

- Restate intended table grain/state. 2) Identify the first version/layer where the control diverges. 3) Check
- keys/schema before rewriting code. 4) Reveal only the next structural hint. 5) Protected Review is permitted only after
- a saved attempt.

12. Small independent task

Given four hypothetical commits, identify what a reader at version 2 should see and what it must not see.

13. Validate

Save the input grain, the operation you performed, at least one independent row/value/schema/history control, and the evidence file or command that proves it. If a control differs, diagnose before continuing.

14. Teach it back

In 60-90 seconds explain transaction log, acid and table versions. Include one failure mode and one validation you would show a teammate.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery. Changed retry from the canonical lesson: Changed log: reconstruct which snapshot is visible after one failed and two successful commits.

16. Evidence / mastery + exact next route

You can repeat the changed task without the lesson/review open, validation passes, and you can defend why the result is trustworthy. A percentage score does not infer mastery. Exact next route: DE13-04

DE13-04 - Create, read, update and delete

1. Why it matters

A pipeline that produces correct rows once can still be unsafe if retries duplicate records, writers race, schema drifts silently, history disappears unexpectedly, or maintenance changes table meaning. The engineering skill is to preserve both business correctness and operational evidence.

2. What you are learning

By the end of this lesson you can create, read, update and delete and prove the result with saved evidence rather than a plausible screenshot.

3. Prerequisite / starting state

Prerequisite: DE13-03 mastered with saved evidence.

4. Picture it in your head

CRUD describes logical table changes: create/write an initial table, read a snapshot, update matched rows, and delete rows intentionally. In Delta, updates/deletes create new table versions rather than editing a Parquet file in place as if it were a spreadsheet.

5. New words before we use them

CRUD describes logical table changes: create/write an initial table, read a snapshot, update matched rows, and delete rows intentionally. In Delta, updates/deletes create new table versions rather than editing a Parquet file in place as if it were a spreadsheet. The transaction layer chooses new active files and records the commit.

6. Useful video / source

Source: Databricks Academy — free Delta / Lakehouse training

Assigned scope: Choose the matching Delta/Lakehouse unit

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

Trace v0→v1→v2→v3 in the normal simulator. Identify the changed and deleted business IDs. Then write pseudocode for CREATE, UPDATE and DELETE in Delta SQL or DeltaTable API without copying the reference implementation. Trace create/read/update/delete transitions and reconcile each version row/value controls.

8. Expected result / what you should see

Save the input grain, the operation you performed, at least one independent row/value/schema/history control, and the evidence file or command that proves it. If a control differs, diagnose before continuing.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

- Calling Parquet itself a Delta table.
- Assuming ACID proves business logic is correct.
- Using automatic schema evolution as permission for any upstream change.
- Treating time travel as permanent backup.

DE13-04 - Practice and proof

11. If you get stuck

- Restate intended table grain/state. 2) Identify the first version/layer where the control diverges. 3) Check
- keys/schema before rewriting code. 4) Reveal only the next structural hint. 5) Protected Review is permitted only after
- a saved attempt.

12. Small independent task

Write your own CRUD pseudocode and the validation after each operation.

13. Validate

Save the input grain, the operation you performed, at least one independent row/value/schema/history control, and the evidence file or command that proves it. If a control differs, diagnose before continuing.

14. Teach it back

In 60-90 seconds explain create, read, update and delete. Include one failure mode and one validation you would show a teammate.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery. Changed retry from the canonical lesson: Changed state: design create/update/delete checks for inventory records.

16. Evidence / mastery + exact next route

You can repeat the changed task without the lesson/review open, validation passes, and you can defend why the result is trustworthy. A percentage score does not infer mastery. Exact next route: DE13-05

DE13-05 - Schema enforcement and schema evolution

1. Why it matters

A pipeline that produces correct rows once can still be unsafe if retries duplicate records, writers race, schema drifts silently, history disappears unexpectedly, or maintenance changes table meaning. The engineering skill is to preserve both business correctness and operational evidence.

2. What you are learning

By the end of this lesson you can schema enforcement and schema evolution and prove the result with saved evidence rather than a plausible screenshot.

3. Prerequisite / starting state

Prerequisite: DE13-04 mastered with saved evidence.

4. Picture it in your head

Schema enforcement rejects incompatible writes instead of quietly corrupting a table. Schema evolution is a controlled decision to accept a compatible structural change.

5. New words before we use them

Schema enforcement rejects incompatible writes instead of quietly corrupting a table. Schema evolution is a controlled decision to accept a compatible structural change. “Automatically accept every schema change” is not resilience.

6. Useful video / source

Source: Databricks — Diving into Delta Lake: Transaction Log

Assigned scope: Use matching transaction-log / ACID / time-travel section

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

Compare v1 schema with v2 schema. Explain why adding `quality_band` is a controlled additive change. Then propose one breaking change—such as value becoming free-text—and write the validation that should stop it. Apply one additive schema change and reject one incompatible type-change proposal with a written reason.

8. Expected result / what you should see

Save the input grain, the operation you performed, at least one independent row/value/schema/history control, and the evidence file or command that proves it. If a control differs, diagnose before continuing.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

- Calling Parquet itself a Delta table.
- Assuming ACID proves business logic is correct.
- Using automatic schema evolution as permission for any upstream change.
- Treating time travel as permanent backup.

DE13-05 - Practice and proof

11. If you get stuck

- Restate intended table grain/state. 2) Identify the first version/layer where the control diverges. 3) Check

- keys/schema before rewriting code. 4) Reveal only the next structural hint. 5) Protected Review is permitted only after
- a saved attempt.

12. Small independent task

Classify four schema changes as safe additive, controlled migration, or reject; explain downstream impact.

13. Validate

Save the input grain, the operation you performed, at least one independent row/value/schema/history control, and the evidence file or command that proves it. If a control differs, diagnose before continuing.

14. Teach it back

In 60-90 seconds explain schema enforcement and schema evolution. Include one failure mode and one validation you would show a teammate.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery. Changed retry from the canonical lesson: Changed schema: decide whether adding nullable device_model vs changing amount STRING→STRUCT is safe.

16. Evidence / mastery + exact next route

You can repeat the changed task without the lesson/review open, validation passes, and you can defend why the result is trustworthy. A percentage score does not infer mastery. Exact next route: DE13-06

DE13-06 - Time travel and version history

1. Why it matters

A pipeline that produces correct rows once can still be unsafe if retries duplicate records, writers race, schema drifts silently, history disappears unexpectedly, or maintenance changes table meaning. The engineering skill is to preserve both business correctness and operational evidence.

2. What you are learning

By the end of this lesson you can time travel and version history and prove the result with saved evidence rather than a plausible screenshot.

3. Prerequisite / starting state

Prerequisite: DE13-05 mastered with saved evidence.

4. Picture it in your head

Time travel means reading an earlier committed table version or timestamp while its required files are still retained. History is useful for auditing, debugging and recovery, but it is not a forever backup.

5. New words before we use them

Time travel means reading an earlier committed table version or timestamp while its required files are still retained. History is useful for auditing, debugging and recovery, but it is not a forever backup. VACUUM/retention policies can make old snapshots unreadable after old files are physically removed.

6. Useful video / source

Source: Databricks — Diving into Delta Lake: Transaction Log

Assigned scope: Use matching transaction-log / ACID / time-travel section

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

Read normal version 0 and current version 2. Compare Ravi/C102 tier and total value. Write a debugging note explaining which version introduced the change and what evidence you used. Compare v0 and v2, identify the change version, and state the retention dependency of time travel.

8. Expected result / what you should see

Save the input grain, the operation you performed, at least one independent row/value/schema/history control, and the evidence file or command that proves it. If a control differs, diagnose before continuing.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

- Calling Parquet itself a Delta table.
- Assuming ACID proves business logic is correct.
- Using automatic schema evolution as permission for any upstream change.
- Treating time travel as permanent backup.

DE13-06 - Practice and proof

11. If you get stuck

- Restate intended table grain/state. 2) Identify the first version/layer where the control diverges. 3) Check

- keys/schema before rewriting code. 4) Reveal only the next structural hint. 5) Protected Review is permitted only after
- a saved attempt.

12. Small independent task

Write a recovery plan using history/time travel without claiming it replaces backup.

13. Validate

Save the input grain, the operation you performed, at least one independent row/value/schema/history control, and the evidence file or command that proves it. If a control differs, diagnose before continuing.

14. Teach it back

In 60-90 seconds explain time travel and version history. Include one failure mode and one validation you would show a teammate.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery. Changed retry from the canonical lesson: Changed history: identify the correct earlier version for rollback investigation and retention risk.

16. Evidence / mastery + exact next route

You can repeat the changed task without the lesson/review open, validation passes, and you can defend why the result is trustworthy. A percentage score does not infer mastery. Exact next route: DE13-07

DE13-07 - MERGE, upserts and idempotent incremental loads

1. Why it matters

A pipeline that produces correct rows once can still be unsafe if retries duplicate records, writers race, schema drifts silently, history disappears unexpectedly, or maintenance changes table meaning. The engineering skill is to preserve both business correctness and operational evidence.

2. What you are learning

By the end of this lesson you can merge, upserts and idempotent incremental loads and prove the result with saved evidence rather than a plausible screenshot.

3. Prerequisite / starting state

Prerequisite: DE13-06 mastered with saved evidence.

4. Picture it in your head

MERGE combines source changes with a target using a match key. A common upsert rule is: matched key → update; unseen key → insert.

5. New words before we use them

MERGE combines source changes with a target using a match key. A common upsert rule is: matched key → update; unseen key → insert. Rerun safety depends on deterministic match logic and source uniqueness.

6. Useful video / source

Source: Databricks Academy — free Delta / Lakehouse training

Assigned scope: Choose the matching Delta/Lakehouse unit

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

Use updates.csv and prove one updated ID plus one inserted ID. Run the simulator twice and prove the second run does not add a sixth row. Then inspect duplicate_source.csv and explain why source-key uniqueness is a precondition. Run normal MERGE simulator, rerun it, and prove row/value idempotency; then trigger duplicate-source diagnosis.

8. Expected result / what you should see

Save the input grain, the operation you performed, at least one independent row/value/schema/history control, and the evidence file or command that proves it. If a control differs, diagnose before continuing.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

- Calling Parquet itself a Delta table.
- Assuming ACID proves business logic is correct.
- Using automatic schema evolution as permission for any upstream change.
- Treating time travel as permanent backup.

DE13-07 - Practice and proof

11. If you get stuck

- Restate intended table grain/state. 2) Identify the first version/layer where the control diverges. 3) Check
- keys/schema before rewriting code. 4) Reveal only the next structural hint. 5) Protected Review is permitted only after
- a saved attempt.

12. Small independent task

Design an idempotent upsert contract: key, source uniqueness check, matched action, unmatched action, rerun proof.

13. Validate

Save the input grain, the operation you performed, at least one independent row/value/schema/history control, and the evidence file or command that proves it. If a control differs, diagnose before continuing.

14. Teach it back

In 60-90 seconds explain merge, upserts and idempotent incremental loads. Include one failure mode and one validation you would show a teammate.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery. Changed retry from the canonical lesson: Changed clinic data: prove MERGE rerun safety and reject duplicate source key P206.

16. Evidence / mastery + exact next route

You can repeat the changed task without the lesson/review open, validation passes, and you can defend why the result is trustworthy. A percentage score does not infer mastery. Exact next route: DE13-08

DE13-08 - Slowly Changing Dimension Type 2 with Delta

1. Why it matters

A pipeline that produces correct rows once can still be unsafe if retries duplicate records, writers race, schema drifts silently, history disappears unexpectedly, or maintenance changes table meaning. The engineering skill is to preserve both business correctness and operational evidence.

2. What you are learning

By the end of this lesson you can slowly changing dimension type 2 with delta and prove the result with saved evidence rather than a plausible screenshot.

3. Prerequisite / starting state

Prerequisite: DE13-07 mastered with saved evidence.

4. Picture it in your head

SCD Type 2 preserves attribute history instead of overwriting it. When a tracked attribute changes, expire the old dimension row and insert a new current row with a new effective period.

5. New words before we use them

SCD Type 2 preserves attribute history instead of overwriting it. When a tracked attribute changes, expire the old dimension row and insert a new current row with a new effective period. Business keys identify the entity across versions; surrogate/version rows represent historical states.

6. Useful video / source

Source: Databricks Academy — free Delta / Lakehouse training

Assigned scope: Choose the matching Delta/Lakehouse unit

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

Open `scd2_history.json`. Find the changed business key. Verify the old row is closed and the new row is current. State two invariants: one current row per key; historical intervals do not overlap. Validate the generated SCD2 history for one current row per business key and non-overlapping periods.

8. Expected result / what you should see

Save the input grain, the operation you performed, at least one independent row/value/schema/history control, and the evidence file or command that proves it. If a control differs, diagnose before continuing.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

- Calling Parquet itself a Delta table.
- Assuming ACID proves business logic is correct.
- Using automatic schema evolution as permission for any upstream change.
- Treating time travel as permanent backup.

DE13-08 - Practice and proof

11. If you get stuck

- Restate intended table grain/state. 2) Identify the first version/layer where the control diverges. 3) Check
- keys/schema before rewriting code. 4) Reveal only the next structural hint. 5) Protected Review is permitted only after
- a saved attempt.

12. Small independent task

Design SCD2 rows for a customer whose tier changes twice; prove one current row remains.

13. Validate

Save the input grain, the operation you performed, at least one independent row/value/schema/history control, and the evidence file or command that proves it. If a control differs, diagnose before continuing.

14. Teach it back

In 60-90 seconds explain slowly changing dimension type 2 with delta. Include one failure mode and one validation you would show a teammate.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery. Changed retry from the canonical lesson: Changed employee dimension: create non-overlapping SCD2 intervals after department change.

16. Evidence / mastery + exact next route

You can repeat the changed task without the lesson/review open, validation passes, and you can defend why the result is trustworthy. A percentage score does not infer mastery. Exact next route: DE13-09

DE13-09 - Bronze, Silver and Gold layer contracts

1. Why it matters

A pipeline that produces correct rows once can still be unsafe if retries duplicate records, writers race, schema drifts silently, history disappears unexpectedly, or maintenance changes table meaning. The engineering skill is to preserve both business correctness and operational evidence.

2. What you are learning

By the end of this lesson you can bronze, silver and gold layer contracts and prove the result with saved evidence rather than a plausible screenshot.

3. Prerequisite / starting state

Prerequisite: DE13-08 mastered with saved evidence.

4. Picture it in your head

Medallion architecture is a design pattern, not three magical folders. Bronze preserves raw/replayable source truth plus ingestion metadata.

5. New words before we use them

Medallion architecture is a design pattern, not three magical folders. Bronze preserves raw/replayable source truth plus ingestion metadata. Silver applies reusable cleaning, typing, deduplication and quality rules at a declared grain.

6. Useful video / source

Source: Databricks Academy — free Delta / Lakehouse training

Assigned scope: Choose the matching Delta/Lakehouse unit

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

Use normal controls: compare `bronze_rows`, `silver_rows` and `gold_value`. Explain why Bronze can contain both base and update events while Silver represents current-state records. Reconcile Gold total back to Silver before trusting it. Map raw events to Bronze, current clean state to Silver, and region aggregate to Gold; reconcile totals.

8. Expected result / what you should see

Save the input grain, the operation you performed, at least one independent row/value/schema/history control, and the evidence file or command that proves it. If a control differs, diagnose before continuing.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

- Calling Parquet itself a Delta table.
- Assuming ACID proves business logic is correct.
- Using automatic schema evolution as permission for any upstream change.
- Treating time travel as permanent backup.

DE13-09 - Practice and proof

11. If you get stuck

- Restate intended table grain/state. 2) Identify the first version/layer where the control diverges. 3) Check

- keys/schema before rewriting code. 4) Reveal only the next structural hint. 5) Protected Review is permitted only after
- a saved attempt.

12. Small independent task

Define Bronze/Silver/Gold contracts for clinic events, including grain, required fields, quarantine and reconciliation.

13. Validate

Save the input grain, the operation you performed, at least one independent row/value/schema/history control, and the evidence file or command that proves it. If a control differs, diagnose before continuing.

14. Teach it back

In 60-90 seconds explain bronze, silver and gold layer contracts. Include one failure mode and one validation you would show a teammate.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery. Changed retry from the canonical lesson: Changed support-ticket pipeline: reconcile Bronze events to Silver latest-state to Gold SLA summary.

16. Evidence / mastery + exact next route

You can repeat the changed task without the lesson/review open, validation passes, and you can defend why the result is trustworthy. A percentage score does not infer mastery. Exact next route: DE13-10

DE13-10 - File layout, compaction and table maintenance

1. Why it matters

A pipeline that produces correct rows once can still be unsafe if retries duplicate records, writers race, schema drifts silently, history disappears unexpectedly, or maintenance changes table meaning. The engineering skill is to preserve both business correctness and operational evidence.

2. What you are learning

By the end of this lesson you can file layout, compaction and table maintenance and prove the result with saved evidence rather than a plausible screenshot.

3. Prerequisite / starting state

Prerequisite: DE13-09 mastered with saved evidence.

4. Picture it in your head

Logical correctness and physical maintenance are separate. OPTIMIZE/compaction can rewrite many small data files into fewer larger files without changing table meaning.

5. New words before we use them

Logical correctness and physical maintenance are separate. OPTIMIZE/compaction can rewrite many small data files into fewer larger files without changing table meaning. Partition choices should follow access patterns and data size; high-cardinality IDs usually make poor partition columns.

6. Useful video / source

Source: Databricks Academy — free Delta / Lakehouse training

Assigned scope: Choose the matching Delta/Lakehouse unit

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

Write a maintenance plan for a table partitioned by a low-cardinality date/region field. Include: symptom, metric, compaction action, validation before/after, retention rule, and rollback/time-travel impact. Design a compaction/partition/VACUUM maintenance checklist that preserves logical controls.

8. Expected result / what you should see

Save the input grain, the operation you performed, at least one independent row/value/schema/history control, and the evidence file or command that proves it. If a control differs, diagnose before continuing.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

- Calling Parquet itself a Delta table.
- Assuming ACID proves business logic is correct.
- Using automatic schema evolution as permission for any upstream change.
- Treating time travel as permanent backup.

DE13-10 - Practice and proof

11. If you get stuck

- Restate intended table grain/state. 2) Identify the first version/layer where the control diverges. 3) Check
- keys/schema before rewriting code. 4) Reveal only the next structural hint. 5) Protected Review is permitted only after
- a saved attempt.

12. Small independent task

Choose a partition key for a 2-year order table and reject order_id; define compaction and retention checks.

13. Validate

Save the input grain, the operation you performed, at least one independent row/value/schema/history control, and the evidence file or command that proves it. If a control differs, diagnose before continuing.

14. Teach it back

In 60-90 seconds explain file layout, compaction and table maintenance. Include one failure mode and one validation you would show a teammate.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery. Changed retry from the canonical lesson: Changed maintenance: diagnose a small-files problem without partitioning by a unique ticket ID.

16. Evidence / mastery + exact next route

You can repeat the changed task without the lesson/review open, validation passes, and you can defend why the result is trustworthy. A percentage score does not infer mastery. Exact next route: DE13-11

DE13-11 - Lakehouse integration and debugging

1. Why it matters

A pipeline that produces correct rows once can still be unsafe if retries duplicate records, writers race, schema drifts silently, history disappears unexpectedly, or maintenance changes table meaning. The engineering skill is to preserve both business correctness and operational evidence.

2. What you are learning

By the end of this lesson you can lakehouse integration and debugging and prove the result with saved evidence rather than a plausible screenshot.

3. Prerequisite / starting state

Prerequisite: DE13-10 mastered with saved evidence.

4. Picture it in your head

Debug a lakehouse from contracts outward: source receipt → Bronze landing → Silver validity → Gold reconciliation → transaction/history evidence → physical layout. Do not start by randomly deleting checkpoints or VACUUMing.

5. New words before we use them

Debug a lakehouse from contracts outward: source receipt → Bronze landing → Silver validity → Gold reconciliation → transaction/history evidence → physical layout. Do not start by randomly deleting checkpoints or VACUUMing. Use table history and control totals to locate the first bad boundary.

6. Useful video / source

Source: Databricks Academy — free Delta / Lakehouse training

Assigned scope: Choose the matching Delta/Lakehouse unit

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

Take the unseen practical dataset. Write a six-step incident runbook that identifies the first bad layer, uses row/value controls, inspects history, checks source-key uniqueness before MERGE, and preserves evidence before repair. Debug a seeded mismatch using layer-by-layer controls and table history without destructive cleanup.

8. Expected result / what you should see

Save the input grain, the operation you performed, at least one independent row/value/schema/history control, and the evidence file or command that proves it. If a control differs, diagnose before continuing.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

- Calling Parquet itself a Delta table.
- Assuming ACID proves business logic is correct.
- Using automatic schema evolution as permission for any upstream change.
- Treating time travel as permanent backup.

DE13-11 - Practice and proof

11. If you get stuck

- Restate intended table grain/state. 2) Identify the first version/layer where the control diverges. 3) Check
- keys/schema before rewriting code. 4) Reveal only the next structural hint. 5) Protected Review is permitted only after
- a saved attempt.

12. Small independent task

Diagnose a scenario where Gold revenue drops but Bronze receipt count is normal; state the first three checks in order.

13. Validate

Save the input grain, the operation you performed, at least one independent row/value/schema/history control, and the evidence file or command that proves it. If a control differs, diagnose before continuing.

14. Teach it back

In 60-90 seconds explain lakehouse integration and debugging. Include one failure mode and one validation you would show a teammate.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery. Changed retry from the canonical lesson: Changed incident: trace a schema-drift failure from Silver back to source and preserve the failed attempt evidence.

16. Evidence / mastery + exact next route

You can repeat the changed task without the lesson/review open, validation passes, and you can defend why the result is trustworthy. A percentage score does not infer mastery. Exact next route: Proceed to the module practical/Gate route after the module mastery checklist.

Module mastery checklist + revision quick reference

- ☐ Every lesson has a saved genuine independent attempt.
- ☐ Every lesson has at least one independent validation/control appropriate to the tool.
- ☐ Protected reviews were opened only after attempts; any weak lesson has a changed retry after review was closed.
- ☐ You can teach back each lesson in your own words and name one realistic failure mode.
- ☐ Required module practical/retest evidence is saved and no critical defect is unresolved.
- ☐ D+1, D+4, D+10 and D+30 revision events are scheduled for weak or high-value competencies.

Revision cadence

D+1: fresh micro-task. D+4: mixed retrieval. D+10: changed/failure case. D+30: interview/project retrieval. Always attempt from memory before reopening a source.

Release boundary

This book is a static learner artifact. It does not prove live tool/cloud execution or a protected Gate pass. Runtime evidence must be recorded separately when the curriculum requires it.